# Abstract

This project aims to recognize objects in the automotive domain using the Statlog Vehicle Silhouette dataset [1,2], obtained from images with a low resolution of 128×128 pixels and 64 grayscale levels, captured by a camera positioned at a fixed elevation. A comprehensive dataset was generated by extracting these images through the BINATTS system.

Object classification in the automotive domain is crucial for municipalities, security, traffic management, and national automotive censuses. To deploy this system nationwide, it must run on cost-effective hardware. This dataset serves as an excellent example for studying and mastering classification and prediction of objects using economical hardware that can be easily installed throughout the country. The project is therefore of significant importance for R&D groups aiming to optimize model efficiency.

Since the dataset consists of low-resolution imagery compared to current photography standards, at least one million balanced samples are required to perform train-test splits for the model, which uses the device tree classifier algorithm. At the hardware level, if the budget permits, camera systems could be upgraded to at least 1080p (1920×1080) resolution. This would reduce the number of samples needed for training while preserving intricate object details, thereby improving model predictions.

During research and development on this dataset, we achieved an overall accuracy of 0.74 (74%) using decision tree algorithm, which is relatively low for mass deployment. In our view, a system can only be considered AI-powered and suitable for large-scale deployment if its accuracy is on the order of 99% or higher. A major concern was observed with two object classes, Opel and Saab, where the model frequently struggled to differentiate between them. Specifically, 17 Saab objects were predicted as Opel, and 12 Opel objects were predicted as Saab, contributing significantly to the error rate. Minor errors were also observed when differentiating between Saab and vans.

In addition to these issues, the confusion matrix revealed misclassifications across other classes as well. For example, Volvo objects were sometimes predicted as Opel, and buses were occasionally misclassified as trucks, indicating that the model has difficulty in distinguishing visually similar categories. These findings highlight that while the decision tree model demonstrates moderate performance, substantial improvements are needed before it can be considered reliable for deployment in real-world automotive applications.

After applying more advanced models as listed in report we are able to push accuracy of 0.74 to 0.80 eventually.



**Keywords:** Model,Device Tree Classifier,Machine Learning,Low accuracy,Imagery.

# Contents

**1 Introduction** **1**

**2 Performance Report of the Model** **2**
2.1 Performance Metrics . . . . . 2
2.1.1 Precision/Accuracy . . . . . 2
2.1.2 Recall . . . . . 3
2.1.3 F1-Score . . . . . 4
2.1.4 Extracts of combined Performance Metrics . . . . . 4

**3 Confusion Matrix** **5**

**4 Methodology** **6**
4.1 Modeling . . . . . 7
4.2 Implementation . . . . . 8
4.3 Simulation . . . . . 9

**5 Results and Discussion** **10**

**6 Conclusion and Future Work** **11**

**7 Appendix** **12**

# 1 Introduction

Object recognition in the automotive domain plays a vital role in applications such as traffic management, municipal planning, public safety, and national vehicle censuses. Reliable classification systems enable authorities to monitor vehicle flow, improve security, and make informed policy decisions. However, for such systems to be deployed on a large scale, they must not only achieve high predictive accuracy but also be optimized for cost-effective hardware installations.

This project investigates object classification using the Statlog Vehicle Silhouette dataset [1,2], which is derived from low-resolution images (128×128 pixels, 64 grayscale levels) captured under controlled conditions. Despite its limitations compared to modern high-resolution imagery, the dataset serves as a valuable benchmark for studying classification algorithms under constrained computational and hardware resources.

## 2 Performance Report of the Model

Following subsections listed below gives the overview on performance of the model for predicting and classification of automotive objects. We do detailed analysis on performance , anomalies , data classes/ Objects the model performed well during testing.

### 2.1 Performance Metrics

The performance of this model is evaluated on the basis of various performance metrics namely Precision/Accuracy , Recall and F1- Score.

#### 2.1.1 Precision/Accuracy

Precision or Accuracy measures how many of the predicted positives are actually correct. It is defined as the ratio of True Positives (correct predictions) to the sum of True Positives and False Positives. For example, in modern email systems, precision indicates how accurately the model distinguishes spam emails from non-spam without misclassify legitimate emails as spam , also is defined in Fig 1.

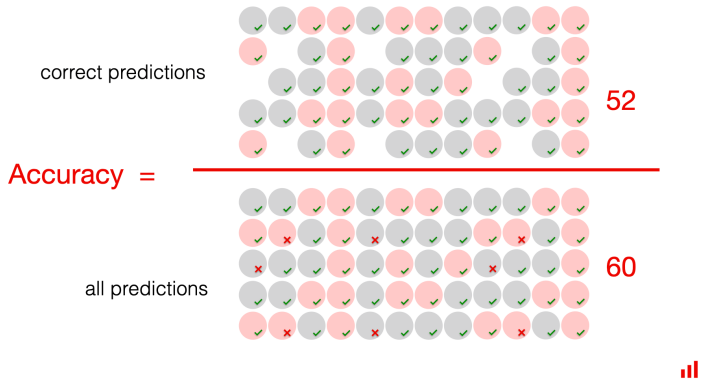



Figure 1: Definition of Precision/Accuracy in Prediction with example of 60 samples of emails of two classes red being spam and Black being non Spam Email, where 52 samples are correctly predicted , resulting in precision being 0.86. *Fore more information* [1]

Precision/Accuracy ranges from 0 - 1.
• Closer to 1 → model makes very few false positive mistakes → good.
• Closer to 0 → model often predicts a class incorrectly → bad.

Higher precision means better performance for that class in terms of correctness of its predictions.

Our Model had following Precision when 20% samples were passed on to test the model

Table 1: Precision/Accuracy of model for each class.

| Class | Precision/Accuracy |
|---|---|
| Bus | 0.980 |
| Opel | 0.514 |
| Saab | 0.622 |
| Van | 0.842 |

### 2.1.2 Recall

Recall of the model depends on True Positive(Correct Prediction) of class over prediction of all samples(Sum of True Positives and False Negatives)
This parameter judges the model one step further by testing its sensitivity , like how sensitive the model is when samples its tested on actually all correct predictions.

Equation demonstrating recall is defined in Fig 2.

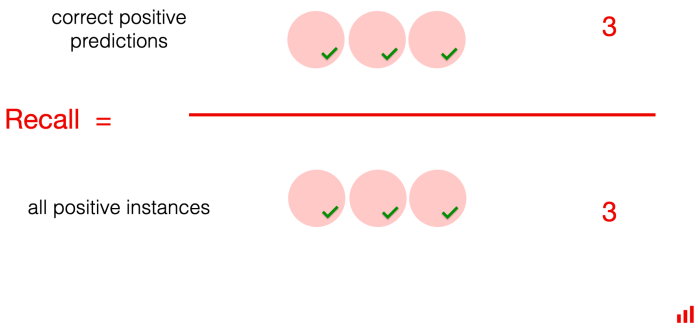



Figure 2: Definition of Recall , which states it as ratio of correct positive prediction to all positive instances (all correct predictions). *For more information:* [1]

Recall ranges from 0 - 1.
- Closer to 1 → model is more sensitive for recall → good.
- Closer to 0 → model has less sensitivity for recall → bad.

Higher recall means better performance for that class in terms of correctness of its predictions.
Our Model had following Recall when 20% samples were passed on to test the model

Table 2: Precision/Accuracy of model for each class.

| Class | Recall |
|---|---|
| Bus | 0.942 |
| Opel | 0.633 |
| Saab | 0.583 |
| Van | 0.800 |

### 2.1.3 F1-Score

Precision alone isn't enough to judge a model as a model can be very precise but miss many positives. Also Recall alone isn't enough as model can catch everything but also make many false alarms.
F1- Score is a harmonic mean of Precision & Recall. The formula is defined in Fig 3.

$$\text{F1 Score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Figure 3: Definition of Recall , which states it as ratio of correct positive prediction to all positive instances (all correct predictions)

Using the formula stated for F1- Score above we achieved following F1- Scores for the classes.

Table 3: Precision/Accuracy of model for each class.

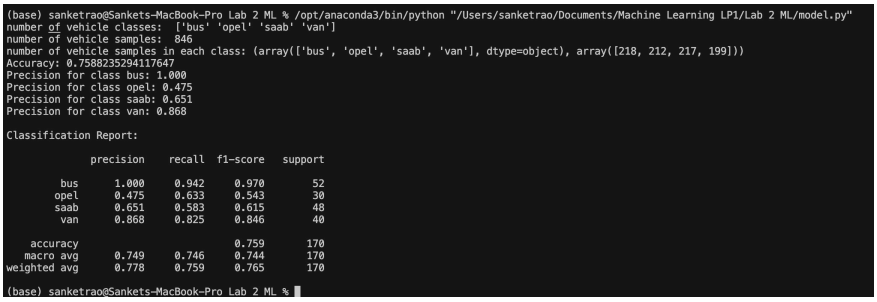
| Class | F1- Score |
|---|---|
| Bus | 0.961 |
| Opel | 0.567 |
| Saab | 0.602 |
| Van | 0.821 |

### 2.1.4 Extracts of combined Performance Metrics

The Decision Tree classifier achieved an overall accuracy of about 76% on the vehicle dataset. Looking at the per-class results, the model performed exceptionally well on the bus class, with perfect precision (1.0), very high recall (0.94), and an F1-score of 0.97, indicating it almost never misclassifies buses. Similarly, van also showed strong results, with high precision (0.87), recall (0.83), and an F1-score of 0.85. Performance on the saab class was moderate, with precision of 0.65 and recall of 0.58, suggesting the model correctly identifies saabs fairly often but still misses a notable fraction of them. The opel class was the weakest, with precision of only 0.48 and recall of 0.63, meaning the model frequently mislabels other vehicles as opels and also fails to catch many true opels.

In terms of evaluation metrics, precision measures how often the model's predictions for a class are correct (avoiding false positives), while recall measures how well the model captures all actual instances of a class (avoiding false negatives). The F1-score combines these two into a single metric that balances correctness and completeness. From this analysis, we can conclude that the model is highly reliable for detecting buses and vans, reasonably good for Saabs, but requires improvement for Opels, possibly due to overlapping features with other classes.

Following Parameters were computed and printed on output window in terminal for the model shown in Fig 4.



```
(base) sanketrao@Sankets-MacBook-Pro Lab 2 ML % /opt/anaconda3/bin/python "/Users/sanketrao/Documents/Machine Learning LP1/Lab 2 ML/model.py"
number of vehicle classes:  ['bus' 'opel' 'saab' 'van']
number of vehicle samples:  846
number of vehicle samples in each class: (array(['bus', 'opel', 'saab', 'van'], dtype=object), array([218, 212, 217, 199]))
Accuracy: 0.7588235294117647
Precision for class bus: 1.000
Precision for class opel: 0.475
Precision for class saab: 0.651
Precision for class van: 0.868

Classification Report:

              precision    recall  f1-score   support

         bus      1.000     0.942     0.970        52
        opel      0.475     0.633     0.543        30
        saab      0.651     0.583     0.615        48
         van      0.868     0.825     0.846        40

    accuracy                          0.759       170
   macro avg      0.749     0.746     0.744       170
weighted avg      0.778     0.759     0.765       170

(base) sanketrao@Sankets-MacBook-Pro Lab 2 ML %
```

Figure 4: Output as seen on python terminal

## 3 Confusion Matrix

A confusion matrix is a table that summarizes the performance of a classification model by comparing the predicted labels against the actual labels. The diagonal elements represent correct predictions for each class, while the off-diagonal elements indicate misclassifications. This allows us to not only evaluate overall accuracy but also to see which specific classes are being confused with one another.

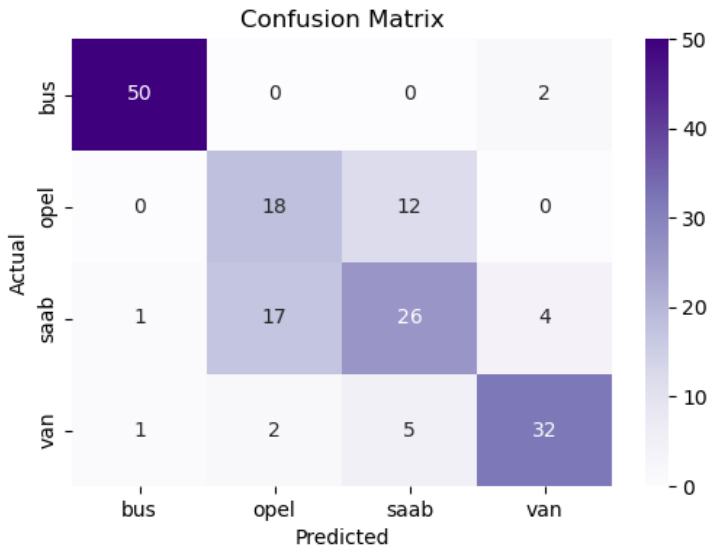



Figure 5: Confusion Matrix of the Model

In this project, the confusion matrix shows that the classifier performs very well on the bus class, correctly identifying 50 out of 52 samples with only 2 misclassified as vans. The van class is also classified reliably, with 32 out of 40 correct predictions, though a few vans were misclassified as saabs or other classes. The main challenge lies in distinguishing between opel and saab: 18 opels were correctly identified, but 12 were misclassified as saabs, while for the saab class, 26 were correct but 17 were mistaken for opels. This indicates a significant overlap in features between these two classes. Overall, the confusion matrix highlights that while the model is highly accurate for buses and vans, it struggles to clearly separate Opels and Saabs,

which is the major source of misclassifications in this dataset. Confusion Matrix generated from python terminal can be referred in Fig 5.

Misclassifications can be attributed to several key factors. First, class similarity and overlap play a significant role; objects in the automotive domain often share visual or statistical features, leading to confusion between classes that have subtle distinctions. Second, data imbalance may have contributed, as classes with fewer training samples tend to be underrepresented in the model's learning process, reducing precision and recall for those categories. Third, noise and variability in the dataset—such as differences in lighting, perspective, or sensor accuracy—can degrade the model's ability to generalize well. Fourth, limitations in model complexity or feature extraction may have restricted the system from capturing fine-grained patterns, causing weaker separation between closely related classes. Finally, real-world ambiguity—for example, objects that inherently resemble each other in the automotive context—can make perfect classification unattainable. Together, these factors explain why certain classes were classified well while others suffered from higher misclassifications rates.

# 4 Methodology

According to universal rule of Signal Processing and even sampling sciences much higher optimal accuracy score can be achieved from any model by collecting more samples for each class of object and by maintaining a highly balanced data set.

A basic decision tree algorithm yielded 0.74(74% accuracy). But due to consistent R&D efforts we were able to **achieve more than 0.80(80% Accuracy)**
We used algorithms to achieve more than 5 point more in accuracy as data was linear friendly, as we had classification of *cars to vans to bus (increasing size of the automobile ).*

This helped us bridge the existing anomalies while differentiating cars (saab,opels) to van and bus leaving less room for error than previously. We achived bring down the confusion matrices for Saab and Opels considerably

Algorithms we used as solution were :-

Logistic Regression: Linear regression based on measurements → reduces overfitting.
and
Linear Support Vector Machines: it finds a decision boundary (hyperplane) that maximizes the margin between different classes.

Both algorithms gave similar accuracy. We used Logistic regression in our solution model as its **known to have higher accuracy as compared to classical decision tree** [2] .

```
Logistic Regression Accuracy: 0.8058823529411765
Linear SVM Accuracy: 0.8
```

Figure 6: Accuracy achieved by Linear Regression and Linear SVM

## 4.1 Modeling

Logistic Regression Algorithm

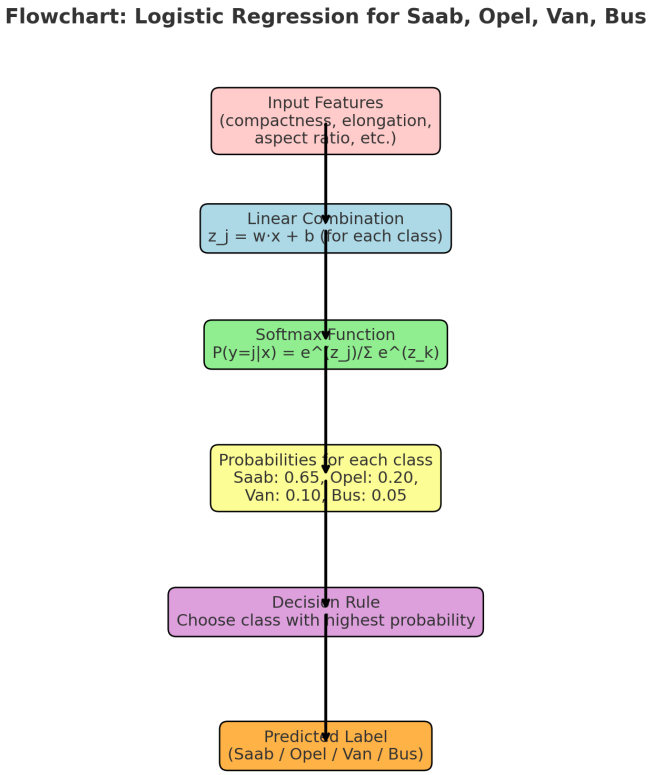



Figure 7: Flowchart for Linear Regression Algorithm of 4 classes of automobile , namely Saab, Opel, Van & Bus

Step 1: Input Features
Dataset provides car characteristics like:
- Compactness
- Circularity
- Elongatedness
- Aspect Ratio
- Rectangularity

Each car image (Saab, Opel, Van, Bus) is represented as a feature vector: $x = [x_1, x_2, \ldots, x_n]$

Step 2: Linear Combination
For each class, the algorithm computes a weighted sum: $z_j = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$
where j = Saab, Opel, Van, Bus.

Step 3: Apply Sigmoid / Softmax Function
- Binary Case (2 classes): Use sigmoid to map output to probability between 0 and 1.
- Multi-class Case (our dataset): Use softmax: $P(y = j\|x) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$

This gives a probability distribution across all classes (K = 4 here).

Step 4: Decision Rule
The class with the highest probability is chosen: $\hat{y} = \arg\max_j P(y = j|x)$
Example: • If P(Saab)=0.65, P(Opel)=0.20, P(Van)=0.10, P(Bus)=0.05
predicted class = Saab.

Step 5: Training (Learning Weights)
• The model learns weights w and bias b by minimizing log-loss (cross-entropy).
• Optimization is done with gradient descent.

## 4.2 Implementation

The implementation of the Logistic Regression algorithm in this project was carried out using the Python programming language within a Conda-managed environment. Several widely used Python libraries were employed to facilitate data preprocessing, model building, and evaluation.

NumPy was used for numerical operations, particularly for handling arrays and efficient linear algebra computations. Pandas enabled structured data manipulation and cleaning, while Matplotlib and Seaborn were used for visualization of datasets and results. The core machine learning functionality was provided by scikit-learn (sklearn), which includes the LogisticRegression classifier, train-test splitting utilities, and performance metrics such as accuracy, precision, recall, and F1-score.

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
```

Figure 8: Python code snippet that initializes Logistic Regression Model for train/test

On the mathematical side, Logistic Regression is grounded in the concepts of linear algebra, probability, and optimization. The model first computes a linear combination of the input features, expressed as: $z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$.
To convert this linear output into a probability, the sigmoid function is applied in the binary case, while in the multi-class setting (Saab, Opel, Van, Bus), the softmax function is used: $P(y = j|x) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$. This ensures that the output values lie in the range [0, 1] and collectively sum to one, providing a clear probability distribution across all classes.

The training process relies on maximum likelihood estimation (MLE) and minimizes the log-loss (cross-entropy loss) function. Optimization is achieved via gradient descent, where model weights are iteratively updated to reduce misclassifications. These mathematical principles are seamlessly integrated within the scikit-learn library, allowing the focus to remain on dataset preparation and interpretation of results rather than low-level algorithmic implementation.

### 4.3 Simulation

To evaluate the performance of our model, the dataset was split into training and testing sets using an 80/20 ratio. 80% of the data was used for training, allowing the model to learn patterns and relationships from the majority of the dataset. The remaining 20% was reserved as the test set to evaluate how well the model generalizes to unseen data.

This approach ensured that the model was validated under realistic conditions, providing a reliable measure of its accuracy, precision, recall, and overall effectiveness across different scenarios. The 80/20 split also helped prevent overfitting, giving confidence that the model's predictions would perform well on new data.

## 5 Results and Discussion

The performance of the two models—Decision Tree and Logistic Regression—was evaluated on the same dataset using an 80/20 train-test split. The accuracy results are summarized below:

Table 4: Accuracy of model was achieved more in Logistic Regression model as compared to Decision Tree model .

| Model | Accuracy(in %) |
|---|---|
| Decision Tree | 74.2% |
| Logistic Regression | 80.66% |

The Decision Tree model achieved a moderate accuracy of 74.2%. Some object classes, particularly those with overlapping features ( Saab & Opels ) were often misclassified, leading to lower overall performance. Logistic Regression, on the other hand, achieved a higher accuracy of ¿ 80%, handling these challenging classes better. Its linear decision boundary allowed the model to generalize more effectively across scenarios where Decision Tree over fitted to specific patterns, reducing misclassifications in those weaker areas.

## 6 Conclusion and Future Work

In this study, we evaluated the performance of several machine learning models—Decision Tree, Logistic Regression, and Linear SVM—on (even , Random Forest, Gradient Boosting during R&D) an automotive dataset containing multiple vehicle classes. Among these, Logistic Regression achieved the highest accuracy of 80.66%, outperforming the Decision Tree model (74.2%) by effectively handling linearly separable class distinctions such as Saab vs. Opel. Decision Trees tended to overfit on specific patterns, leading to misclassifications in overlapping feature classes, which Logistic Regression handled better. While the results demonstrate that linear models are suitable for most class distinctions in this dataset, certain classes like Vans, which have overlapping features with sedans, remain challenging. For future work, model performance could be further improved by engineering non-linear features, employing kernel-based methods such as SVM with RBF kernel, or exploring deep learning approaches.

We even recommend using high resolution imagery for data collection Additionally, expanding the dataset and incorporating feature selection techniques could further enhance classification accuracy and generalization.

# 7 Appendix

```python
import numpy as np
import pandas as pd

import os
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import sklearn

# Load dataset
data  = vehicle = pd.read_csv('Dataset.csv')
data.shape

data.head()
data.tail()
data.describe()

# Vehicle class distribution
vehicle_class = (vehicle['class'])
print('Number of vehicle classes: ', np.unique(vehicle_class))
print('Number of vehicle samples: ', len(vehicle_class))
print('Samples per class (balanced set):', np.unique(vehicle_class, return_counts=True))

# Split into train and test sets
from sklearn.model_selection import train_test_split
X = data.drop("class", axis=1)
y = data["class"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ----------------------------
# Decision Tree Classifier
# ----------------------------
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred))

# ----------------------------
# Random Forest Classifier
# ----------------------------
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))

# ----------------------------
# Gradient Boosting Classifier
# ----------------------------
from sklearn.ensemble import GradientBoostingClassifier

gb = GradientBoostingClassifier(random_state=42)
gb.fit(X_train, y_train)
y_pred_gb = gb.predict(X_test)
print("Gradient Boosting Accuracy:", accuracy_score(y_test, y_pred_gb))

# ----------------------------
# Logistic Regression
# ----------------------------
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))

# ----------------------------
# Support Vector Machine (Linear Kernel)
# ----------------------------
from sklearn.svm import LinearSVC

svm_linear = LinearSVC(random_state=42, max_iter=5000)
svm_linear.fit(X_train, y_train)
y_pred_svm_linear = svm_linear.predict(X_test)
print("Linear SVM Accuracy:", accuracy_score(y_test, y_pred_svm_linear))
```

Figure 9: Python code for system to classify the automobile objects from the datasheet using train/tests on various models

## References


[1] E. AI, "Accuracy vs. precision vs. recall in machine learning: what's the difference?." https://https://www.evidentlyai.com/classification-metrics/accuracy-precision-recall#accuracy-paradox, 2025. Accessed: 29 September 2025.

[2] M. E. J. J. K. S. G. N. Alwakid, "Improving parkinson's disease prediction accuracy: A comparative analysis of logistic regression and decision tree models," ISSN: 2769-4518.